

Chapter 3

Triggering and Offline Software

In Run3 of the LHC, the increased luminosity leads to an average of 60 collisions per bunch crossing (termed "Pileup") (REF?). Given that these bunch crossings occur at a rate of 40MHz (every 25ns) and can create a large number of individual particles, the processing and storage facilities are incapable of saving and analyzing every interaction generated in each bunch crossing. In fact, storing all of the raw data at this rate of 40MHz would generate up to 40 TB/s of data [19]. The triggering system, therefore, is designed to drastically reduce the number of output events while preserving as many events matching signal signatures as possible. Overall, the triggers aim to reduce the data rate from 40MHz down to just 7kHz. This reduction is achieved by a two-tiered trigger system consisting of the Level 1 (L1) triggers and the High Level Triggers (HLT). Since there are many possible signal signatures depending on the analysis being conducted, there are many individual triggers at both L1 and HLT. Details of the L1 triggers will be given below in section 3.1, and the HLT will be described in great detail in section 3.2.

3.1 Level-1 Triggers

The Level 1 trigger is described in detail in [11], but the primary points are summarized in this section. As described in section 2.1.2, collisions in CMS during Run 3 occur at a rate of 40 million per second (40MHz). The goal of the L1 trigger is to select at most 110,000 of these events per second (110kHz) to pass on to the next level of the trigger, the HLT. The L1 trigger is

designed to operate as quickly as possible, with a target latency of just 4 μ seconds. In order to achieve this very low latency, minimal event reconstruction is done at L1; instead, this trigger performs minimal processing on the so-called trigger primitives (TP), which are the raw outputs from detector elements. The details of these TPs will be discussed below in section 3.1.1.

In order to process events as quickly as possible, software is eschewed in favor of an FPGA-based hardware system [13]. There are two of these systems corresponding to the two major detector systems: the L1 calorimeter trigger processes the information from both the Electron and Hadron Calorimeters, while the L1 muon trigger processes the information from the muon chambers. The calorimeter trigger system is of the greatest interest to this paper, as the photon candidates from $H \rightarrow \gamma\gamma$ events are largely measured by the ECal with some additional information also being drawn from the HCal. In section 3.1.2, then, the calorimeter trigger system will be discussed in detail.

These two systems consist of many individual triggers, known as the L1 menu. This menu is designed to target specific physics objects, and consists of several triggers for each object. Muon triggers comprise nearly half of this menu. Additional triggers exist for both τ leptons and the identification of jet objects. Finally there are triggers designed to measure electrons and photons (termed e/γ triggers). These triggers are, of course, the most important for $H \rightarrow \gamma\gamma$ processes, and the identification of photon candidates is the first step in finding and measuring these signal processes. The details of the L1 menu, with particular emphasis on the e/γ triggers, are presented below in section 3.1.4.

3.1.1 Information Available to L1 Triggers

The inputs to the L1 trigger are entirely in the form of trigger primitives. For the calorimeter trigger, these TPs are in the form of energy deposition in a specific detector element [11]. The HCal and ECal, detailed in section 2.2.3 and 2.2.2, are only able to measure the energy deposited in their individual elements. So-called Trigger Towers (TTs) group together the ECal and HCal elements in a specific region of a detector. For the barrel detector, a 5x5 set of EB

crystals is combined with the corresponding HB tower behind them into a single TT. This gives a barrel resolution of $\Delta\eta \times \Delta\phi = 0.087 \times 0.087$ radians. In the endcap, the presence of the HF detector complicates the TTs; the information from the EE, HE, and HF detectors must all be merged in a single TT per channel. Additionally, the geometry is complicated by the variable crystal size and spacing in the endcap. While the overall TT resolution consequentially varies in the endcap region, the maximum TT size is $\Delta\eta \times \Delta\phi = 0.17 \times 0.17$ radians.

The L1 TPs for the muon triggers are considerably more complex, as there are more detector elements to consider. The cathode strip chambers, drift tubes, and resistive plate chambers described in section 2.2.4 all overlap and must be combined in each TT. Furthermore, the TPs in the muon L1 system are not merely energy depositions. Instead, the muon detector subsystems provide energy, coordinate, and quality information that is later used to reconstruct a track.

3.1.2 The L1 Trigger Architecture

The calorimeter L1 trigger consists of two distinct layers of processing. Layer-1 receives the inputs directly from the TPs listed above, and sorts and calibrates them rapidly. Layer-2 takes the outputs from Layer-1 and performs primitive calibration and reconstruction, which allows the trigger to roughly identify physics objects such as electrons, τ leptons, and jets. A rather complex time-multiplexed hardware system is used in Layer-2, which allows the low, fixed latency necessary at the L1 trigger. Layer-2 consists of several separate reconstruction algorithms, of which primarily the e/γ reconstruction algorithm is of interest for the $H \rightarrow \gamma\gamma$ trigger. Details of this reconstruction will be provided in the next section. Once the objects are reconstructed, they are passed to the next stage in the trigger system.

The muon L1 trigger has 3 separate muon track finders (MTFs) which aim to reconstruct tracks in three distinct regions of the detector (barrel, endcap, and overlap). In the barrel, only the DT and RPC TPs are used to reconstruct tracks. In the endcap, only the CSC and RPC TPs are used to achieve the same goal. In the overlap region, all three subdetectors are combined.

These MTFs feed their outputs directly to the global muon trigger (μ GMT), which receives up to 108 muon tracks per event. The μ GMT then sorts the tracks and removes duplicates. The final output of the muon triggers has at most 8 muon tracks.

Both the Layer-2 calorimeter trigger and the μ GMT output to the Global Trigger (μ GT). The μ GT combines this information using a set of complex algorithms called the trigger menu described below in section 3.1.4. Six independent boards are used to distribute the processing of the L1 menu, and a final decision is rendered on whether to keep or discard an event. Below, in figure 3.1, a schematic of the full process from TPs to μ GT is reproduced from [11].

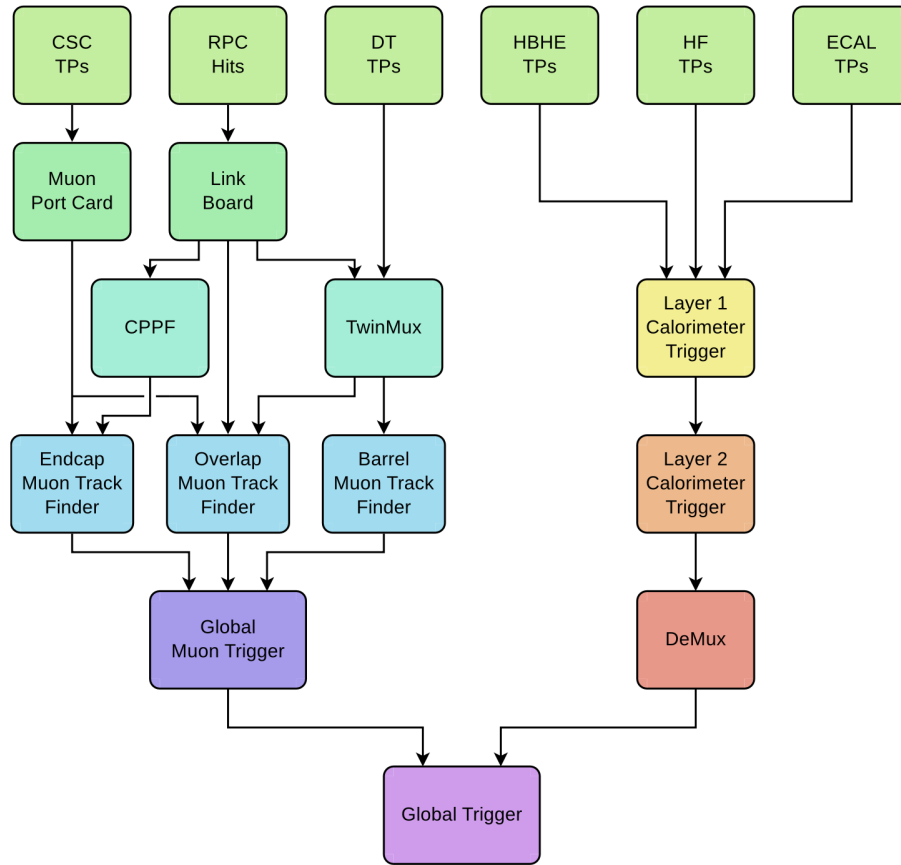


Figure 3.1. A diagram of the L1 Trigger architecture, where the lefthand tree corresponds to the muon triggers and the righthand tree to the calorimeter triggers. Reproduced from [11].

3.1.3 The L1 e/γ Reconstruction

Since photons and electrons have no associated tracking information, the only way that they can be identified is by their energy signatures [11]. The only way to reconstruct e/γ objects, then, is to very carefully reconstruct the shape, size, and magnitude of these energy signatures deposited in the TTs of the ECal and HCal. The major tenet of this is called clustering. Firstly, the "seed tower" is defined as the trigger tower that has the highest energy deposition in a local region. Then, contiguous towers are taken where the energy threshold exceeds $E_T = 1$ GeV. These clusters are created dynamically, allowing different signature shapes for later analysis.

Of course, due to the effects of pileup in the detector elements (as discussed in section (REF?)), care must be taken to consider only those contiguous towers which appear to be part of a genuine e/γ signature. There are a few clustering vetos as the L1 level designed to remove signatures that do not appear to originate from an e/γ candidate: a shape veto is applied to select contiguous clusters that appear to have the correct shape, thereby excluding wider or more irregular pileup showers wherever possible; a fine grain veto bit measures the compactness of the signature in the seed tower and discriminates against hadron-induced showers; an H/E veto measures the ratio of energies deposited in the HCal and ECal, rejecting signatures with higher HCal energy depositions [11].

The next major reconstructed quantity at the L1 trigger is the energy isolation. Essentially, a 6×9 region ($\eta \times \phi$) around the main cluster is defined, and the total energy deposited in this region is measured and compared against a chosen threshold for the specific L1 trigger. So, then, a higher isolation energy means that the cluster may not be a clean individual e/γ signature and instead could come from a larger hadronic shower or from pileup energy deposition. This isolation energy will be a major component of the HLT and offline reconstructions, as well. The components of the L1 clustering algorithm are shown in figure 3.2, reproduced from [11].

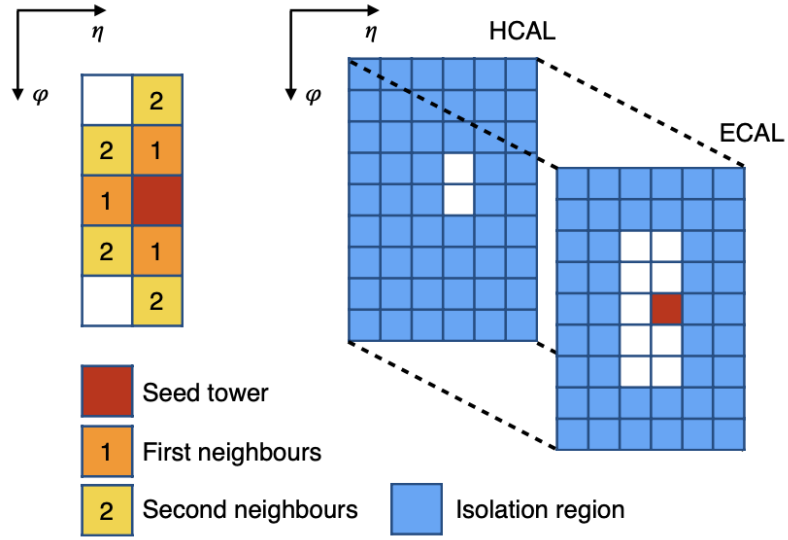


Figure 3.2. A diagram of an L1 e/γ cluster. The central seed tower is shown in red, and the clustered contiguous towers are shown in orange and yellow. The isolation region is shown in blue. Reproduced from [11].

The full energy of the cluster, excluding the isolation region, is taken as the "raw energy" of the event and is then compared to various thresholds imposed by different L1 triggers. The L1 trigger only has access to this raw energy, the general shape of the energy shower, and the isolation energy. While these quantities will be further refined in later, more sophisticated steps of reconstruction, they still prove powerful enough to reduce the L1 trigger rate to the required value without discarding an unacceptable amount of possibly genuine e/γ signatures.

3.1.4 The L1 Menu

The full L1 consists of a suite of many individual triggers combined together, which is termed an L1 menu. While several different menus exist for different physics programs and for testing/calibration, the Global Trigger (GT) menu is the standardized full menu used for normal p-p collisions. This menu consists of the full suite of hadronic, heavy lepton, and e/γ triggers. The percentage of the 100kHz rate allowed for each type of trigger in Run 2 is shown in figure 3.3. This information has not been updated for Run 3, but the overall structure of the L1 menu has not changed. Clearly, e/γ triggers are a major component of the full L1 menu; combining

both single and double e/γ triggers gives a total percentage of 31.2%.

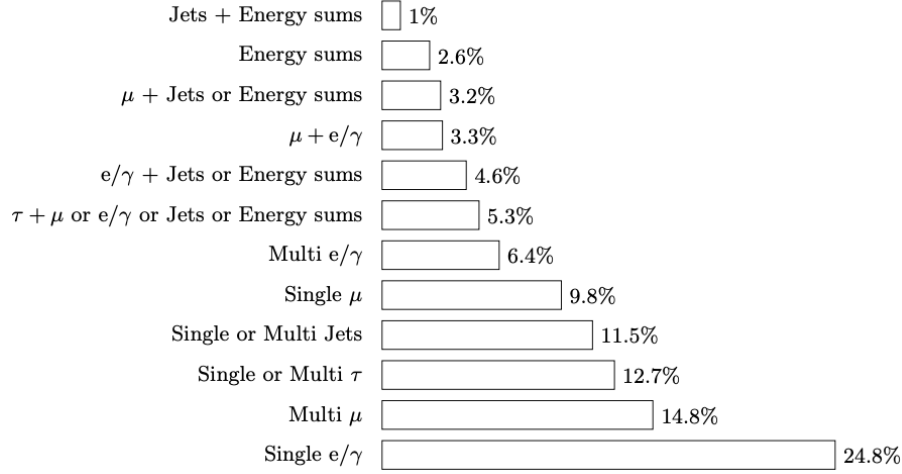


Figure 3.3. A comparison of the percentage of the total rate allowed for each category of trigger. Reproduced from [11].

Of these classes of triggers, the single and double e/γ triggers are the most relevant to the $H \rightarrow \gamma\gamma$.

3.2 High Level Trigger

The final step of triggering before data is output for analysis, the HLT aims to reduce the rate from the 110kHz output by the L1 trigger down to a total of just 5kHz [10]. In contrast to the hardware-based L1 trigger, the HLT is a software-based suite of triggers that runs on a dedicated server of processors called the HLT filter farm. While the L1 trigger menu drastically reduces the overall data rate (by a factor of 400), the HLT is aimed specifically to obtain the best possible events to be provided to analyses. As the last stage before samples are saved and further reconstructed before analysis, the HLT must aim to produce as pure of signal samples as possible. So, then, the signal efficiency of the HLT is of vital importance to every analysis at the CMS. Of course, the details described in this section will be of vital importance to an understanding of the development of a BDT-Based HLT for $H \rightarrow \gamma\gamma$. Sections 3.2.1-?? will give a general picture on the reconstruction, rate/timing considerations, and software structure of the HLT as a whole.

Section 3.2.4, however, will discuss the details of the current standard HLT for $H \rightarrow \gamma\gamma$.

3.2.1 HLT Event Reconstruction

Event reconstruction poses both a major challenge and a major strain on the computational resources at the LHC. As a result, reconstruction gets progressively more sophisticated at each stage of event rate reduction [6]. So, after the primitive reconstruction of the L1 allows that trigger menu to reduce the rate by a factor of approximately 400, the HLT is designed to perform a more thorough reconstruction of the remaining events to help achieve the necessary high signal efficiency. The HLT reconstruction starts with the L1 candidate (termed a L1 seed), and aims to more carefully reconstruct the signature and produce additional quantities derived from the values recorded at L1.

While the L1 attempts to construct a single cluster as if it came from a single photon or electron, the reality of the detector produces additional complications. In fact, many photons and electrons produced at the interaction vertex will either undergo bremsstrahlung (radiation from the curving of a charged particle) or pair production (the creation of an e^+/e^- pair from an energetic photon) [6]. As a result, one e/γ particle produced from a primary interaction may present as several (often overlapping) signatures in the ECal. Since multiple clusters may overlap, the first step is to determine how much of each energy deposit in a single crystal should be associated with each cluster. A gaussian signature is assumed, centered at the cluster seed. Then, depending on the number of overlapping clusters, an algorithm associates a share of the energy deposited in the crystal with each cluster. At this stage, there is a collection of well-resolved clusters in close proximity.

The most important step of e/γ reconstruction is termed superclustering, and it is the process by which neighboring (or overlapping) clusters are associated together into one final supercluster (SC) that contains as much of the signature from the original e/γ object as possible [6]. Superclustering is a two-part algorithm that grows in complexity. Firstly, a "mustache algorithm" uses the basic information from the ECal crystals and the preshower detector to create

an initial candidate for a supercluster. Then, a much more complex "refined algorithm" uses information from the trackers to attempt to determine the origin of a specific cluster and ensure that it should be considered part of the supercluster.

The mustache algorithm is so termed because it looks for clusters associated together in a geometric region that resembles a mustache shape. As with the L1 cluster creation, an original seed is chosen as the center of the supercluster. Now, however, the seed is an entire resolved cluster as opposed to a single crystal. Then, two geometric quantities are constructed for each of the surrounding clusters: $\Delta\eta = \eta_{seed-cluster} - \eta_{cluster}$ measures the η distance, and $\Delta\phi = \phi_{seed-cluster} - \phi_{cluster}$ measures the ϕ distance to the cluster from the central seed. The details of the mustache shape are determined by the algorithm, and depend on the E_T of the seed cluster since charged particles with higher E_T will be bent less by the magnetic field and therefore will have smaller signatures in the ECal. An example of the mustache algorithm is shown in figure 3.4, where electrons produced with $1 < E_T^{seed} < 10 GeV$ are simulated in the region $1.48 < \eta_{seed} < 1.75$.

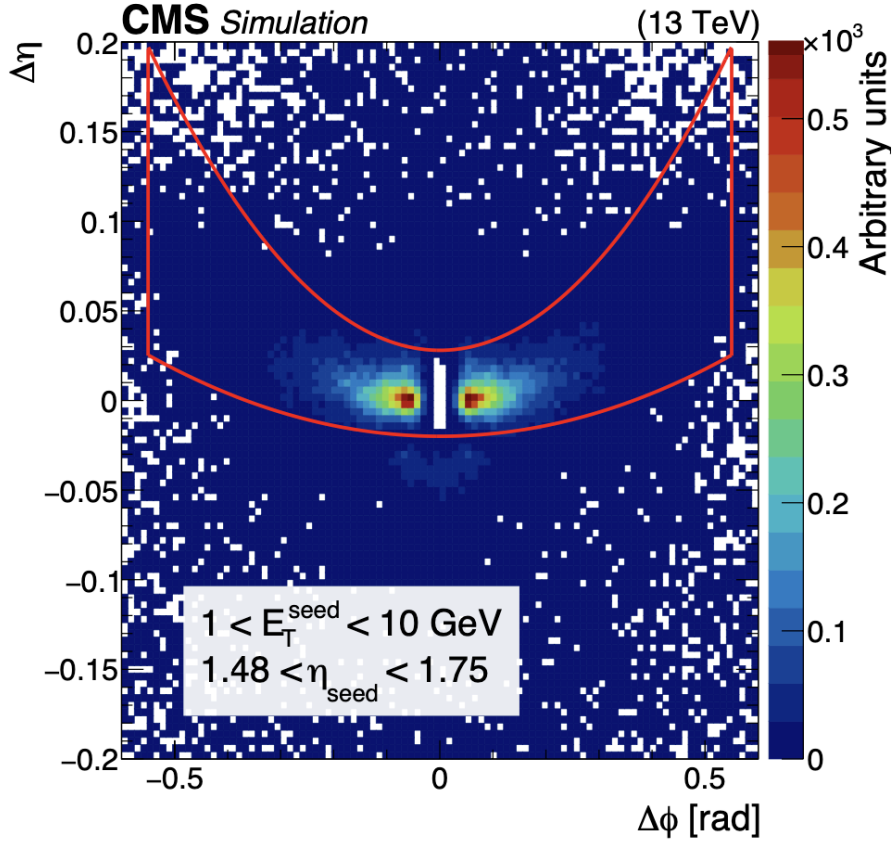


Figure 3.4. A distribution of the $\Delta\eta$ versus $\Delta\phi$ for the superclustering of simulated electrons. The z-axis represents the number of clusters matched with the simulation around the seed. The red line represents the set of clusters selected by the mustache algorithm. The seed cluster can be seen as the white region in the center of the plot (where both $\Delta\eta$ and $\Delta\phi$ are equal to 0). Reproduced from [6].

After the mustache algorithm is applied using only the ECal crystals and preshower detector, the much more robust "refined algorithm" applies complex tracking information to further refine the superclustering. The track reconstruction will be described in detail later in section 3.3.1, where the differences between the HLT and offline reconstruction will be elucidated. Once the tracks have been constructed, they are used to separate clusters originating from both bremsstrahlung and photon conversion. Bremsstrahlung – the release of photons due to the bending of an electron track in the magnetic field – is relatively easy to quantify. Since electrons will have tracks associated with them, the trajectory of the GSF (REF?) track can be measured at each layer of the tracker. Bremsstrahlung photons will be emitted tangentially to the

curving electron, so a "bremsstrahlung tangent" can be constructed at each tracker hit to show the direction of such a photon. This tangent can then be linked to a compatible ECal cluster, if one exists.

Photons originating from the primary vertex very often convert to an electron-positron pair. In fact, simulations show that the fraction of photons undergoing conversion can be as high as 60% where the tracker material is thickest in front of the ECAL [4]. While the above superclustering is usually able to identify conversion pairs from their resulting shower shape, some events may be recovered by use of the tracking information for the electron-positron pair. In some cases, however, only one of the two legs of the conversion pair may be reconstructed. In order to still reconstruct and preserve these events, a BDT is employed to associate possible conversion tracks to ECal clusters.

Once a supercluster is constructed and the relevant clusters are added together, various quantities can be reconstructed at the trigger level. Measures of the energy, the shape of the ECal showers, and various isolation measurements can be constructed by using the ECal energy signatures and the tracking information obtained. These variables will be described in more detail in section 3.3.1.

3.2.2 HLT Trigger Rates and Timing

In order to appraise the online HLT performance, two major metrics are used: the trigger rate gives a direct measure of how many events the trigger will accept, and the trigger timing measures how long it takes for a decision to be rendered for each event. These two metrics strike at the heart of the major considerations of the HLT; it is vital for storage space that the HLT reduces the total number of saved events, and it is vital for the CPU processing time that each event can be accepted or rejected quickly.

The limit for the total rate of the HLT in Run 3 is 7kHz [19]. This total rate is composed of 2.5kHz of data processed immediately (prompt data) and 4.5kHz stored for later processing (data parking). Of course, this is the entire rate of the whole HLT menu. As with the L1 trigger

menu, there are many types of physics analyses comprising the HLT menu. Figure 3.5 shows an example of the HLT rate allotted to each physics group. This plot is reproduced from [10], and is for Run2 2018 data. While the trigger rate and instantaneous luminosity will be higher for Run 3, the general proportion of rate allotment still holds. Only 12.1% of the trigger rate is dedicated to Higgs processes, of which there are many. As a result, the overall data rate specifically for $H \rightarrow \gamma\gamma$ analyses will be considerably lower than this. So, any proposed trigger development must be carefully monitored to ensure it does not exceed the rate budgeted to the corresponding analysis.

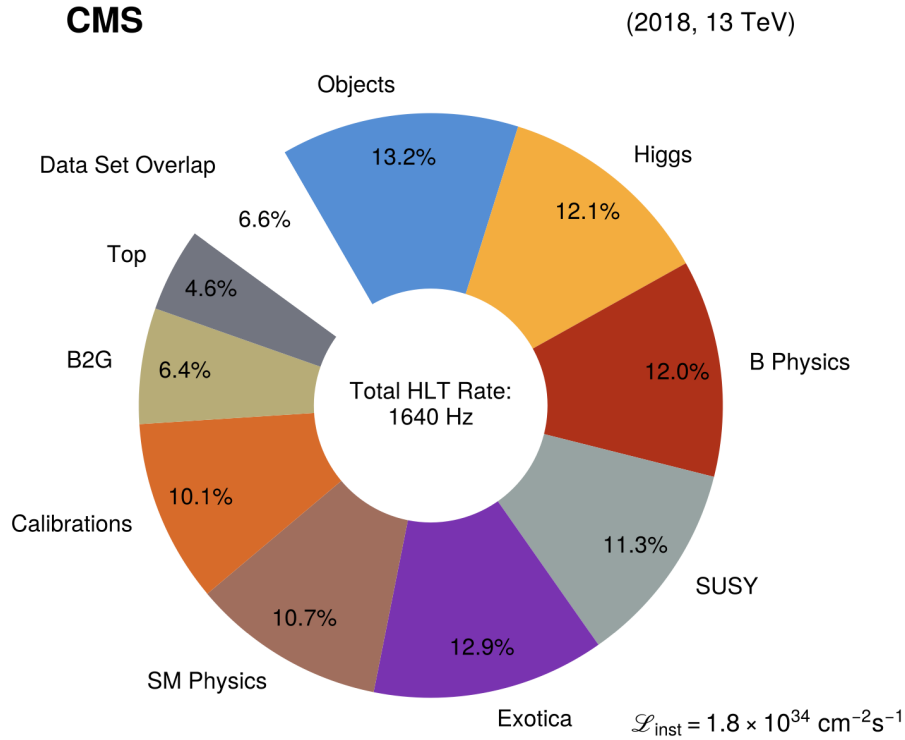


Figure 3.5. A pie chart showing the percentage of the full rate allotted for each physics object group for 2018 data. Note that this example of rate shares operates at a lower instantaneous luminosity and trigger rate than the Run3 data will have. Reproduced from [10].

In addition to parking and prompt data streams, "scouting" data streams are taken at a very high rate but with highly compressed information. In early Run 3, these scouting streams reached an average of 26kHz of rate [1]. These streams only save HLT-level information, and

use a simple reconstruction in order to capture rare physics processes at the trigger level using very high rate.

Since many of the events at the trigger level are processed in real time, the amount of time taken per event is a key metric. This timing includes all of the necessary HLT-level reconstruction, as well as the application of whatever cuts and calculations that each individual HLT requires. Since 110kHz of events are passed from the L1 trigger during a standard run period, it is vital that the processing time per event is carefully monitored. As with the trigger rate, any proposed addition to the HLT menu must be verified to have little to no impact on the overall processing time per event. Since much of the time spent processing each event goes towards reconstruction and tracking algorithms, and individual HLT is not likely to noticeably impact the total processing time unless it is egregiously inefficient.

Moving from Run2 into Run3, the overall data rate has consistently increased year over year. As the storage and processing capabilities have risen, the total allowed rate of the prompt, parking, and scouting datasets has also risen. Additionally, due to improvements to the hardware used to perform the HLT reconstruction (namely, the addition of GPUs in addition to CPUs used in many algorithms), the processing time per event has actually decreased considerably during Run3. Figure 3.6, reproduced from (REF? LHCP PAPER), shows both of these effects in detail. At the left, the rates of the various data streams as well as the instantaneous luminosity is shown as a function of the run year. At the right, the total timing is shown for both CPU-only and CPU+GPU processing for the 2022 running year. Of particular interest for this paper is the 2024 run period, as this will be used for the results given in chapter 7.

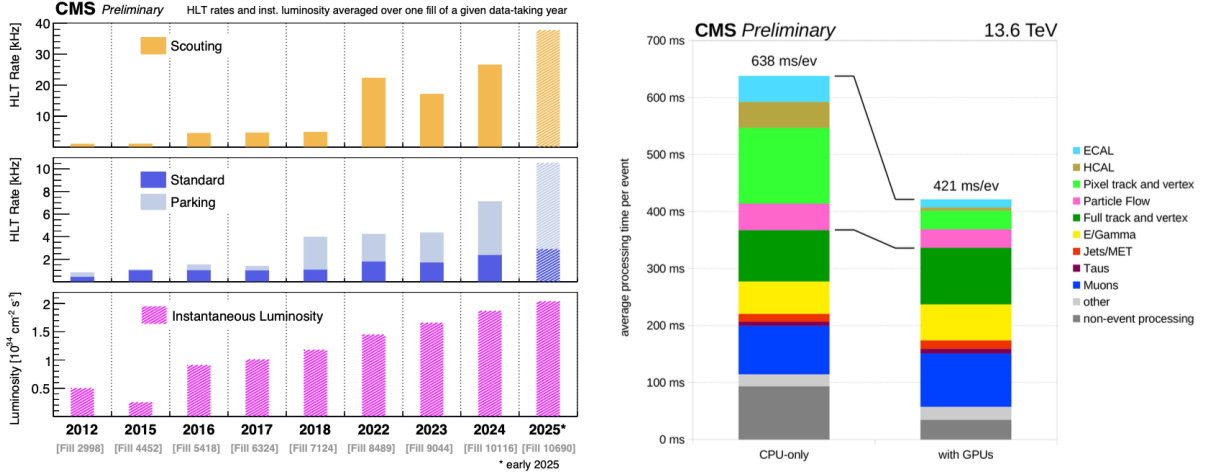


Figure 3.6. On the left, the rates of the scouting dataset (top), prompt (middle, solid), and parking (middle, hatched) data streams are shown. Additionally, the instantaneous luminosity is shown (bottom). To the right, the menu timing is shown with CPU-only and CPU+GPU processing for the 2022 running year. Reproduced from (REF? LHCP PAPER)

3.2.3 HLT Software Framework

The HLT software is built within the CMS Software (CMSSW) framework. Since this is the same software framework used for many future steps of analysis, this provides a relatively standardized set of tools and functions by which the HLT can be constructed. The HLT is of a modular design, where individual modules can be written once and then used by multiple HLTs; many tools, such as the reconstruction of certain quantities or the application of cuts on a specific variable, will be used by a large range of HLTs within the full menu.

There are two major types of HLT modules; EDProducers and EDFilters. An EDProducer module performs a specific step of the reconstruction, starting with a specific prerequisite input and producing a new value as an output. These EDProducers are what allow the input information being passed from the L1 trigger to be reconstructed and combined into more complex variables. For the $H \rightarrow \gamma\gamma$ triggers, EDProducers begin with the e/γ candidates constructed at the L1 and produce reconstructed showershape and isolation variables sequentially. These EDProducers vary greatly in their complexity; while some producers perform simple calculations to construct new quantities, many apply complex algorithms and MVA methods to reconstruct objects like

tracks.

After a quantity has been produced by an EDProducer, cuts are often applied to the newly constructed variable using EDFilters. These EDFilters are considerably more simple than EDProducers – instead of outputting an event, they simply remove candidates that do not pass the selection from the event. An EDFilter can make different cuts on events in the barrel and endcap ECal detectors.

The general structure of the HLT menu is a sequence of EDProducers interspersed with EDFilters. To reduce processing time as much as possible, simple quantities are usually calculated first and cut on, so that the next producer has fewer candidate inputs for which to calculate more complex quantities [6]. Luckily, a software package called ConfDB exists to more easily handle this construction of an HLT path. Modules (EDProducers and EDFilters) are defined within CMSSW, but then are combined together into sequences and full paths in a graphical interface that allows simple insertion and rearrangement of modules. This allows preexisting modules to effortlessly be deployed in a new path, minimizing the amount of additional software development necessary to insert a new path into the menu.

3.2.4 Current Standard HLT (sHLT) for $H \rightarrow \gamma\gamma$

Currently, standard $H \rightarrow \gamma\gamma$ analyses rely on a single trigger to accept events for the analysis. This trigger will be termed the sHLT for the remainder of this dissertation. This diphoton trigger is designed to look for a pair of photons that could be constructed to originate from the decay of the Higgs boson. This trigger depends largely on cuts on the shower shape, isolation, and kinematics of each photon. After these single photon cuts are applied, possible combinations of the remaining photons are constructed and checked against a final diphoton invariant mass requirement [3].

Before applying any HLT cuts, the sHLT begins by only considering events passing certain L1 triggers. Specifically, only events passing either the single or double e/γ triggers are passed onto the next modules. The single e/γ L1 triggers generally have higher E_T requirements

on the single photons, whereas the double e/γ L1 triggers have looser requirements on the E_T of each photon. The full trigger consists of a combination of two filters. The first "leg" requires a photon candidate to pass a single e/γ L1 trigger with an $E_T > 30\text{GeV}$ cut. This L1 seeded leg is the tighter of the two cuts. Then, a second "leg" does not require the tight $E_T > 30\text{GeV}$ L1 seed. Instead, it makes an independent cut on $E_T > 22\text{GeV}$. This leg is referred to as the unseeded leg. The final HLT cut requires one photon passing the seeded leg, and another passing the unseeded leg. Each of these filters makes requirements on the shower shape and isolation of each photon candidate.

The first shower shape cut applied is to the R_9 . Since this measure of the supercluster shape does a great job excluding non-photon signatures, a hard cut on it allows many QCD-like signatures to be immediately discarded; if a sufficiently large amount of the super cluster energy is not in the central 9 crystals, the signature is considerably less likely to originate from a photon. The cuts are placed at $R_9 > 0.5$ in the barrel and $R_9 > 0.8$ in the endcap. The values on this cut are different for the two detector regions to account for the different crystal geometries. Next, a hard cut on the H/E is applied. It is expected that hadronic jets will have considerably higher energy deposition in the HCal than the ECal, so large values of H/E are more likely to correspond to non-photon particles. So, then, this cut is placed at $H/E < 0.12$ in the barrel and $H/E < 0.1$ in the endcap.

The rest of the shower shape and isolation cuts depend on the R_9 of the photon candidate. If a photon candidate is of high R_9 (above 0.85 in the barrel and 0.9 in the endcap), no further cuts are applied. Generally, signatures with such a high R_9 are very likely real photon candidates. Below these thresholds, though, additional cuts are applied. Firstly, cuts of $\sigma_{i\eta i\eta} < 0.015$ and $\sigma_{i\eta i\eta} < 0.035$ are applied in the barrel and endcap, respectively. After this, an ECal isolation cut is applied at $\mathcal{I}_{ECal} < 6\text{GeV}$. Taken together, these cuts allow a photon candidate with poor R_9 to be preserved if the supercluster is well-isolated with a reasonable shower shape. Finally, to the unseeded leg filter, an additional track isolation cut is applied at $\mathcal{I}_{Track} < 6\text{GeV}$.

Finally, these two filter legs are combined into one final filter. This filter also applies

Table 3.1. Table showing the cuts applied by the standard diphoton HLT for $H \rightarrow \gamma\gamma$. The track isolation cut is only applied to the unseeded second leg.

Region	R_9	H/E	$\sigma_{i\eta i\eta}$	$\mathcal{I}_{ECal}$ (GeV)	$\mathcal{I}_{Track}$ (GeV)
Barrel	[0.5,0.85]	< 0.12	> 0.015	< 6.0	< 6.0
	> 0.85	< 0.12	–	–	–
Endcap	[0.5,0.9]	< 0.15	> 0.035	< 6.0	< 6.0
	> 0.9	< 0.15	–	–	–

a minimum mass requirement at $M_{\gamma\gamma} > 90\text{GeV}/c^2$. So, then, the final HLT requires that two photons (one with $E_T > 30\text{GeV}$, another with $E_T > 22\text{GeV}$) pass the showershape and isolation cuts, and that their combined invariant mass is greater than $90\text{GeV}/c^2$. These showershape and isolation cuts are tabulated in table 3.1.

3.3 Offline Software

After the triggers are processed, referred to as "online" processing, there is still much work to be done. As will be discussed in sections 3.3.1 and 3.3.2, considerable reconstruction can be done "offline" after the triggers filter events and the partially-reconstructed samples are saved to storage. Whereas the online processing is primarily concerned with reducing the trigger rates as much as possible while preserving maximum signal, the offline processing is aimed at creating final analysis examples with as complete reconstructions and corrections as possible. Additionally, offline processing is aimed at reducing overall file sizes and processing times for future stages in the analysis, as discussed in section 3.3.3.

3.3.1 Offline e/γ Reconstruction

Many of the details of the offline reconstruction have already been given in section 3.2.1, as the offline and online reconstruction are designed to be as similar as trigger-level resources will allow. The superclustering performed at the HLT is still performed offline, as this is a relatively lightweight algorithm that provides great power to the HLT. The major difference for e/γ objects between the two levels of reconstruction, however, lies in the tracking information. Since tracks

must be constructed by complex algorithms combining many tracker hits, this process is largely pared back at the trigger level.

Both the HLT and offline electron track reconstruction are based on a Gaussian Sum Filter (GSF) [2]. Since this process is very CPU-intensive, it is not done for all possible tracker hits. Instead, electron tracks begin with a hit pattern that might correspond to an electron track, called a "seed". These seeds take two forms. ECal-driven seeds start from a mustache SC satisfying specific criteria, then finds tracker hits that may be associated with this seed. All HLT-level tracks are ECal driven, which is one of the major ways that the processing time is kept within the strict limits of that level. Tracker-driven seeding looks at all possible generic tracks and applies algorithmic selections to identify those possibly originating from an electron.

ECal-driven tracker seeds first select a mustache SC which satisfies $E_{SC,T} > 4\text{GeV}$ and $H/E_{SC} < 0.15$, where E_{SC} is the total supercluster energy and H is the energy of all HCal towers within $\Delta R = \sqrt{(\Delta\eta)^2 + (\Delta\phi)^2} = 0.15$ of the center of the SC position [6]. Then, triplets and doublets of inner pixel tracker hits are combined, and the relative positions of these possible tracks in ϕ and z are compared to the ECal SC. The track is then extrapolated back towards the collision vertex using the SC location the seed energy, and the strength of the magnetic field in the region through which the track moves. If the first two hits of a tracker seed are matched to this predicted trajectory from the SC, then the tracker seed is chosen to seed a GSF track. This ECal-driven track seeding is $> 95\%$ efficient for tracks with $E_T > 10\text{GeV}$ [6].

The tracker-driven approach iterates over all possible generic tracks coming from the interaction vertex, and is therefore much less CPU-efficient. An iterative algorithm called the Kalman Filter (KF) is used to reconstruct every possible track. A combination of cut-based selection OR a BDT-based selection criteria are applied to each track, and if it is found to be compatible with a ECal cluster it is used as a seed for a GSF track. Given the complexity of this seeding procedure, it becomes clear why this tracking algorithm is not applied in the HLT reconstruction.

Once a collection of ECal- and tracker-driven tracking seeds is created, the actual electron

tracks can be fully reconstructed. Now, the KF algorithm is used iteratively to build the track from each tracker hit beginning from the first tracker layer and moving outward. Each step along the way, the energy loss of the electron due to collisions with the tracker material is modeled. If the next tracker layer has multiple possibly associated hits, up to 5 track trajectories can be built using a χ^2 fit. Conversely, in the case where a single hit may belong to multiple possible tracks, the hit is associated with the track with either the least number of missing hits or the lowest χ^2 value. After the KF algorithm reconstructs a track candidate, a GSF fit is used to estimate its parameters at each tracker layer.

Once a GSF track candidate is obtained, it must be associated conclusively with an ECal cluster to form a final electron candidate. This process is termed track-cluster association [6]. This association is performed using a complex BDT. The BDT is trained on track kinematic and quality variables, SC showershape variables, and track-cluster matching variables. Since this BDT is so complex, it is immediately clear why the number of GSF candidates must be properly reduced before full track reconstruction. The track-cluster association is handled differently for tracker-driven and ECal-driven seeds. For track-driven seeds, a cut on this BDT is used to determine if a track and cluster are associated. For ECal-driven seeds, either the same BDT requirement or cuts of $|\Delta\eta| < 0.02$ and $|\Delta\phi| < 0.15$ are applied, where the differences are between the actual SC location and the location of closest approach to the SC of the extrapolated track.

3.3.2 Particle Flow Reconstruction

Another major difference of the offline reconstruction is the existence of the particle-flow (PF) algorithm. At its most basic level, the PF algorithm seeks to combine all possible particle information into one cohesive algorithm [9]. PF is able to establish the 4-momenta of each particle in an event, and identify it as a hadron, photon, electron, or muon. This final categorization allows additional quantities to be calculated, and is the full particle reconstruction used for analysis [6]. For e/γ objects, the primary elements to link together are tracks and ECal

clusters. ECal clusters have been discussed in detail in section 3.2.1. Tracking has been discussed in section 3.3.1.

A major piece of information added to the event record at the offline software level is the vertexing information. By tracing a track backwards towards the interaction region, a specific location of collision, called the primary vertex, can be constructed. Reconstructing this vertex is a key part of the full particle flow algorithm. This vertex is constructed using the full tracking information for electrons, and is therefore dependent on the offline track reconstruction discussed above [4].

In order to fully reconstruct an e/γ object, the ECal clusters, full tracking information, and vertexing information must be combined into the PF algorithm. The first step of PF is called the link algorithm designed to link together signatures from the different subdetectors into a single signature. The link algorithm works pairwise, considering possible pairs of signatures in nearby detector elements that may be worth associating. Of course, given the dozens to hundreds of possible detector signatures, this could quickly become overly CPU-intensive. So, the link algorithm only considers elements nearby each other in the (η, ϕ) plane [9]. Several possible links between signatures will be discussed in the next paragraphs. Once links are established, PF blocks are constructed that consist of multiple linked elements fed into the PF algorithm.

(IS TRACK-CLUSTER ASSOCIATION THE SAME AS PF LINK ALGORITHM? I Think so). In addition to the track-cluster associated discussed in section 3.3.1, the PF algorithm must link other elements together. Cluster-cluster linking associates hits within the ECAL with hits in the HCAL. As with the track-cluster linking, a distance is determined in the (η, ϕ) plane in the barrel and the (x, y) plane in the endcaps. If multiple links are possible, the algorithm simply chooses the linked elements with the smallest separation distance. Links are made between tracks if a common secondary vertex can be found between two tracks. These displaced vertices come from interactions of a primary particle with the material, which then can produce two or more associated particles some distance from the primary vertex. If a track between the primary and secondary vertex, corresponding to the primary particle, can be reconstructed, then that track

is linked with the tracks originating from the secondary vertex. Finally, linking between inner tracker tracks and the muon detector can be linked, though that is not relevant for this analysis.

The final PF reconstruction follows a standardized progression [9]. Firstly, possible muons are reconstructed. Since muons will produce a clear signature in the muon detectors where few other particles will be found, this reconstruction is used to remove associated tracker and calorimeter hits from the PF block. Then, the electron reconstruction is performed, aiming to differentiate between electrons and well-isolated photons. Again, tracks and clusters identified by these reconstructions as part of an electron or photon are removed from the PF block. Finally, hadron reconstruction is handled with the remaining signatures. Of course, due to the focus of this paper on $H \rightarrow \gamma\gamma$, only the details of the e/γ reconstruction are of particular interest.

As already mentioned, electrons very regularly emit bremsstrahlung photons, and photons very regularly undergo e^+e^- conversion due to the material in the detector. The e^+e^- from converted primary photons even undergo bremsstrahlung themselves, often. As a result, even at the PF level, electrons and photons are very hard to reconstruct separately. The primary differentiator used by the PF algorithm is the association with a GSF track [9]. If a GSF track can be linked to an ECAL cluster without 3 or more additional tracks involved, then the particle is seeded as an electron candidate. If a ECAL cluster with $E_T > 10\text{GeV}$ cannot be linked with a GSF track, the particle is seeded as a photon candidate.

Then, the HCAL energy deposition within a distance of 0.15 in (η, ϕ) must be less than 10% of the ECAL supercluster energy for both photon and electron candidates. This helps make sure that the signature is not a misidentified hadronic jet, which would deposit much of its energy in the HCAL. After this requirement, ECAL clusters and GSF tracks must be associated with the candidates. Any ECAL cluster that can be associated with the supercluster or with the tangent of a GSF track (thereby being a bremsstrahlung photon) is automatically added to the candidate. Once the ECAL clusters from the PF block are selected, any track linked to these clusters is accepted as long as the energy and momentum deposited in the HCAL is consistent with the electron hypothesis.

The final identification of electron candidates is much more intensive than that for photons. A complex BDT is used to determine whether a chosen electron candidate can be positively identified as an electron. This BDT uses input variables related to the GSF tracks, the KF χ^2 , and the total number of hits [9]. Photon candidates are then checked on two metrics. First, the photon must be well-isolated from other tracks and clusters in the event. Secondly, the showershape and H/E values must be consistent with a photon shower.

3.3.3 Data Formats at Different Reconstruction Levels

Any given sample of either data or monte carlo generated by the CMS experiment pass through a full data flow with subsequent steps of reconstruction and size reduction [20]. Data begins its life in the RAW format, which contains only the information available to the HLT as discussed in section 3.2.1. Then, the RECO format is created, which adds all of the online reconstruction information from the HLT reconstruction to the RAW format. This RECO format serves as the input to the final offline reconstruction described in section 3.3.1. Whereas the RAW data format consists of about 1MB of disk space per event, the RECO format takes about 2-3x as much space [14]. The trigger developments that will be discussed later in chapter 5 are performed on the RAW data format.

The next three stages of data formats contain essentially the same information, but aim to progressively reduce the storage size of the sample. This is achieved in two ways: the total number of events stored in the data can be reduced by a process called "skimming"; the size of each event can be reduced by eliminating event content. The skimming process is generally handled by the trigger system. After the RECO format, event information is summarized and reduced in multiple steps in order to reduce the size of each individual event.

The first data format constructed by CMS was the Analysis Object Data (AOD) tier, first implemented in Run 1 of the LHC. The AOD format reduces the storage size per event by about 85% compared to the RECO form at [14]. This process takes a huge amount of processing time, so it is generally not rerun often. In fact, the AOD processing typically runs immediately after

the data is collected or the MC is generated, and is only rerun every few years as updates to the offline reconstruction or the AOD format are implemented. This AOD format was the primary centrally-produced data format throughout Run 1.

A major design philosophy difference between the RECO (online) and AOD (offline) data formats is the structuring of data into physics objects. After the PF algorithm is applied to an event, it can be structured according to high-level physics objects such as hadronic jets, isolated photons and electrons, and muons [9]. So, then, every event has a collection of possible physics objects for each subsequent data tier. Additionally, very simplified individual particle candidates containing mostly kinematic information determined by the PF algorithm can be saved. This is a fantastic step towards making the data as analysis-friendly as possible; a $H \rightarrow \gamma\gamma$ analysis, for example, can simply concern itself with the collection of photon physics objects and disregard information on jets, muons, and other leptons. So, then, the AOD formats help not only to reduce the required storage space, but additionally reduce the burden on each analysis group by making the data easier to traverse.

For Run 2, with the proliferation of data and MC produced, an updated centrally-produced format was needed to reduce the storage size of samples. This led to the introduction of the MINIAOD format [20]. This format aimed to reduce the per-event storage size by an additional factor of 10 relative to the AOD format. While some event information is lost at this data tier, 95% of physics analyses in Run 2 are able to use the format without issues. The remaining 5% can run private production of custom data formats directly from the AOD. An additional benefit of this process is that the MINIAOD processing can be rerun multiple times a year, allowing for more frequent updates to reconstruction and calibration [14]. The MINIAOD format contains both high-level physics objects (leptons, photons, and jets). It also contains the very basic particle candidates identified by PF, which contain basic kinematic information.

For Run 3, the NANOAO format was introduced to provide an even further reduced storage size. This format is able to reduce the per-event storage to about 1-2kB per event, which is a reduction of a factor of over 1000 compared to the RECO format [14]. The NANOAO is

very regularly changing and being updated as different analysis groups' needs change; processing the NANO AOD from the MINIAOD is a very fast process that can be run without undue strain on computation resources. There is a reduction in the per-event information saved, so in reality about 70% of analyses in Run3 will be able to solely use the NANO AOD format. Other groups are still able to access the MINIAOD format for their specific needs.

The NANO AOD format is the data format used for the final results of this paper shown in chapter 7. The general structure of this data tier is very easy to use and understand. Whereas the MINIAOD format is composed of physics objects that must be accessed with specific CMS software, the NANO AOD format is structured as "trees" that can be directly accessed using standardized ROOT software [14]. Each tree has a collection of "branches" which correspond to specific physics observables. Within each branch, there is an array of all of the values associated with a particular event. So, this event-based architecture allows event information to quickly be access and traversed, leading to much improved ease in analysis compared to previous data formats.

In order to achieve this massive reduction in event size, much of the low-level detector information such as tracks and clusters are dropped from the event. Furthermore, many of the inputs used for identification algorithms are dropped, with the NANO AOD opting to instead keep only the outputs of the identifications [14]. Additionally, many of the high-level physics quantities saved are redundant with the lower-level information used to derive them. So, many of the lower-level information bits that could later be rederived if necessary are omitted from the event collections. If, for some reason or another, it is later deemed necessary to restore any of the information omitted from the NANO AOD, a new version can be updated to include it rapidly.